Extended Abstract

Motivation Writing highly performant GPU kernels (in CUDA) remains a major bottleneck for deploying high-performance AI systems. Domain experts iterate through a *write → run → profile* loop, guided by two verifiable signals: **functional correctness** and **runtime speed-up**, making it a natural target to apply Reinforcement Learning (RL). Recent interest (Ouyang et al., 2025) attempts to leverage Large Language Models (LLMs) to generate CUDA kernels, yet their outputs are brittle and rarely fast. We therefore ask the question: *can RL help LLM learn an iterative write–run–revise workflow and optimize kernels effectively?*

Method We present **Bob**, a model trained with our multi-turn reinforcement-learning training procedure, for generating CUDA kernels. We extends the standard Group Relative Policy Optimization (GRPO) (Shao et al., 2024), by letting model undergo by multiple rounds of refinement during rollout and train on each turn, mirroing the write–run–revise workflow of a human engineer. Each optimization turn contains $n{=}4$ refinement turns. At turn t the context includes the task problem, all prior kernels $k_{<t}$, and their build/exec/correctness/speed feedback. The model proposes a kernel k_t, which is compiled, timed, scored with a score s_t. The generation and kernel k_t at turn t then receives a scalar reward r_t which follows:

$$r_t = \sum_{i=t}^{T} \gamma^{\,i-t} s_i, \quad s_i = 0.3 \cdot \mathbf{1}_{\text{correct}} + \frac{T_{\text{baseline}}}{T_{k_i}} \cdot \mathbf{1}_{\text{correct}}, \ \gamma = 0.4.$$

Note each (intermediate) turn retained as training examples, to address the sparse reward problem for this task and enable better sample efficiency. Each turn uses a reward that aggregates future turns to help model learns how to arrive at better steps in the future.

Implementation Starting with base model `Qwen QwQ-32B`, we apply GRPO on 180 `KernelBench` tasks. We trained on an 8 × NVIDIA H200 node using an asynchronous compile–run queue where we evaluate 16 candidate kernels per problem in parallel and log every refinement turn as its own RL update. After each turn, the verbose CoT is distilled into a short summary to prevent context blow-up while preserving the rationale behind edits. A strict sanitizer zeros the reward for kernels that invoke high-level PyTorch ops or copy the reference implementation, ensuring the policy must deliver genuine CUDA improvements and prevent reward hacking Overall, 40 GRPO global iterations consume ≈ 40 GPU-hours; we evaluate the model under a test-time compute set up with 16 parallel trajectory and 8 refinement steps each.

Results We evaluate Bob (multi-turn RL), single-turn RL (standard GRPO) against other models on 100 unseen KERNELBENCH tasks. Bob improves upon its base model (QwQ-32B (Team, 2025c)) both in correctness (56% → 82%) and mean speedup of generated kernels (in pure CUDA): from 0.53x to 1.10x over PyTorch Eager, while outperforming frontier models like OpenAI o4-mini (0.78x). We note while both multi-turn RL and single-turn RL achieve best@16 correctness of **82 %** it improves kernel performance much more (**1.10x** vs 0.85x for best@16 performance).

Discussion *Why is multi-turn valuable?* We found multi-training helps get around the sparse-reward issue with this hard task and increases sample efficiency. We train on every turn, as a slow-but-correct kernel could be a stepping stone towards an eventual fast one, and the dense intermediate reward with iterative feedback helps navigate the complex search space. We then also explore varying the number of turns during training roll-outs and found 4-turns are necessary. We also experiment with fine-grained rewards: balanced, stability-centric, and performance-centric. We found that the simplest strategy in balanced yielded the best results, whereas the other two strategies resulted in higher risk.

Conclusion Bob shows that equipping an LLM with an explicit *write–run–revise* loop and training it end-to-end with multi-turn RL improves both *functional reliability* and *runtime efficiency* of generated CUDA kernels. Future work can extend beyond CUDA and target other parallel computing backends such as AMD ROCM, explore richer search spaces (e.g. include runtime telemetry), and implementing more sophisticated search methods during training and test-time.

Bob: Multi-Turn RL for Generating CUDA Kernels

Simon Guo
Department of Computer Science
Stanford University
simonguo@stanford.edu

Shafin Khan
Department of Electrical Engineering
Stanford University
shafink@stanford.edu

William Li
Department of Computer Science
Stanford University
willyli@stanford.edu

Abstract

Writing GPU kernels is a challenging task and critical for AI systems' efficiency. It is also highly iterative: domain experts write code and improve performance through execution feedback. Moreover, it presents verifiable rewards like correctness and speedup, making it a natural environment to apply Reinforcement Learning (RL). To explicitly incorporate the iterative nature of this process into training, we develop a flexible multi-turn RL recipe that addresses unique challenges encountered in real-world settings, such as learning from long trajectories and effective reward attribution across turns. We present Bob, a model trained with multi-turn RL for CUDA kernel generation and optimization. In our evaluation setup, Bob shows significant gains over its base model (QwQ-32B), improving correctness of generated kernels (in pure CUDA) from 56% to 82% and mean speedup from 0.53x to 1.10x of baseline (PyTorch Eager), and surpassing frontier models like o4-mini (0.78x).

1 Introduction

Writing efficient GPU kernels (Dao et al., 2022; Zhao et al., 2025; Ye et al., 2025) in domain-specific languages (CUDA (Nickolls et al., 2008), Triton (Tillet et al., 2019), ThunderKittens (Spector et al., 2024), CUTLASS (NVIDIA Corporation, 2025), etc.) is critical for enabling AI systems' efficiency at scale, yet it remains difficult and costly due to the deep domain expertise required. This has led to a surge of interest in exploring how Large Language Models (LLMs) could help generate GPU kernels (Ouyang et al., 2025; Li et al., 2025; NVIDIA, 2025) using agentic systems (Damani et al., 2024; Chen et al., 2025; METR, 2025; Lange et al., 2025; Google DeepMind, 2025) that leverage extensive test-time compute. These inference-based approaches are inherently limited by the base models' capability in this domain. On the other hand, the presence of verifiable rewards in the form of correctness and speedup against a reference implementation makes reinforcement learning (RL) a natural approach. This leads to our investigation: *How can we train a model using RL to solve the real-world engineering task of CUDA kernel generation?*

GPU kernel generation emphasizes not just functional correctness, but more importantly performance — distinguishing this code optimization problem from binary-reward tasks that involve passing unit tests (Jimenez et al., 2024) or producing an acceptable proof (Zheng et al., 2022). Since speedup is a continuous goal, performance engineers take an iterative approach: they conduct many rounds of optimization based on previous kernel code, its execution result, and timing profiles. Hence, arriving at an optimized solution relies on multiple turns conditioned on previous execution feedback. In contrast, popular RL methods to train LLMs on verifiable rewards (Shao et al., 2024; Lambert et al.,

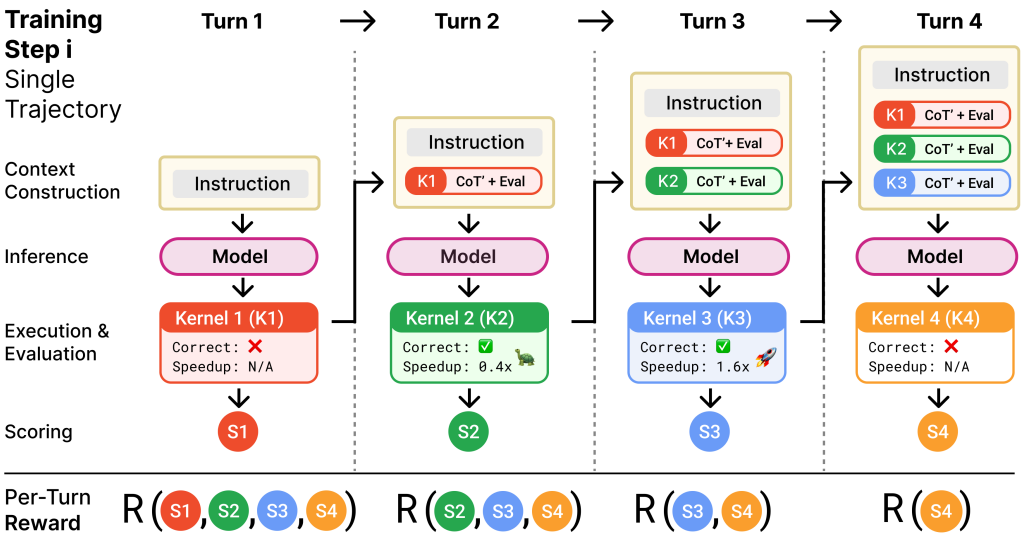

Figure 1: Within each training step, the model iteratively generates, executes, and refines kernels over multiple turns. Kernels are rewarded individually, based both on their performance and their contribution to subsequent speedups: `K1`, for example, while incorrect, leads to both a correct, slow kernel, `K2`, and a correct, performant kernel, `K3`, and should thus be rewarded accordingly. This setup enables Bob to learn advanced code generation strategies that span multiple turns. Note: `CoT'` is the summarized chain of thought (CoT).

2025) rely on the outcome reward of a single turn ("single-turn RL training"). We hypothesize that explicitly incorporating successive turns of code generation, execution, and feedback into each RL training step ("multi-turn RL training") better mirrors the iterative nature of kernel development, helping the model to learn more advanced code generation strategies that span multiple refinement turns.

We design a simple yet effective multi-turn RL training recipe, shown in Figure 1, that addresses the *key challenges* of training for CUDA kernel generation and optimization:

1. **Long trajectories lead to sparse rewards and context explosion**. To improve sample efficiency, we split trajectories and use each turn as an individual training sample. To address context explosion from long CoTs while preserving reasoning information, we summarize CoTs of prior turns.
2. **Finding an optimal solution may require rewarding suboptimal kernels that eventually lead to more performant ones.** Therefore, we study approaches to aggregate intermediate rewards across turns, finding a configuration that balances the correctness-performance trade-off.
3. **Reward hacking is prevalent as kernel generation is an open-ended, real-world engineering task:** e.g. the model can trick the evaluation harness, lazily copying the reference implementation instead of actually implementing kernels. To prevent this, we analyze the model's failure modes and enforce strict rule-based checks.

Enabled by our multi-turn RL training method on 180 KernelBench tasks from Level 1 and 2, we present Bob, a RL-trained model to generate CUDA kernels. We compare Bob and other models in our evaluation setting on a representative KernelBench eval set. Bob improves upon its base model (QwQ-32B (Team, 2025c)) both in correctness (56% $\rightarrow$ 82%) and mean speedup of generated kernels (in pure CUDA): from 0.53x to 1.10x over PyTorch Eager, while outperforming frontier models like OpenAI o4-mini (0.78x).

To summarize, we design an effective yet flexible multi-turn RL training strategy that significantly improves model's capability on CUDA kernel generation. This strategy addresses challenges that arise in real-world settings, and may be applicable to other environments that benefit from iterative optimizations. We found multi-turn trained model outperforms the single-turn trained model across different evaluation setups.

2 Background and Related Work

2.1 LLM for GPU Kernel Optimization

There has been a surge of interest in exploring how to leverage LLMs to generate GPU kernels (NVIDIA, 2025), driven by the high cost and the long engineering cycles required to develop them (e.g. 2 years for efficient FlashAttention (Dao, 2023) port after Hopper GPU release). However, frontier models underperform on representative benchmarks like KernelBench (Ouyang et al., 2025) and TritonBench (Li et al., 2025), likely due to GPU code being severely underrepresented in the training data (CUDA, for example, accounts for less than 0.01% of pretraining data in the Stack (Kocetkov et al., 2022; Li et al., 2023)). Collecting more expert-written code is expensive, as only a limited number of developers are able to implement high-quality kernels. To tackle this task, there has been an explosion of agentic systems (Damani et al., 2024; Chen et al., 2025; METR, 2025) with custom workflows and evolutionary search methods (Lange et al., 2025; Google DeepMind, 2025). Yet these approaches typically incur high inference cost — e.g. $15 per kernel (Lange et al., 2025). Improving the base LLM's kernel-generation ability is therefore essential — and could significantly boost the efficiency for downstream agentic workflows.

2.2 RL Optimization for LLMs Targeting Verifiable Domains

Reinforcement Learning techniques like GRPO (Shao et al., 2024) have been shown to significantly enhance LLMs' performance on verifiable domains (Lambert et al., 2025) such as math (Team, 2025b; Wang et al., 2025b) and competitive programming (Team, 2025c; Luo et al., 2025a,b). These approaches can be further adapted for real-world software tasks, using fine-grain unit tests (Liu et al., 2023) or comparisons between code edits (Wei et al., 2025) as outcome rewards. Existing methods for code optimizations — where objective concerns performance beyond correctness — have been largely confined to supervised fine-tuning (Waghjale et al., 2024) and imitation learning (Shypula et al., 2024), highlighting Bob's RL approach a novel contribution for this setting.

Given that tasks like performance optimization or long-horizon planning require multiple sequences of interrelated actions, several works (Goldie et al., 2025; Cao et al., 2025; Wang et al., 2025a; Zhou et al., 2024; Zhuang* et al., 2025) have explored RL training for multi-turn optimizations beyond optimizing for outcome from a single turn. Specific for the code setting, RLEF (Gehring et al., 2025) frames code generation as a multi-turn RL task: the model is allowed a fixed number of refinements turns and assigned a single binary pass/fail reward for final generation — training with such an approach yields notable sample-efficiency gains. Unlike RLEF, which assigns rewards only at the final turn, our multi-turn RL framework for Bob trains on every turn regardless of how optimal the code is, and optimizes for performance beyond just correctness. It is worth noting that Bob's multi-turn RL training could be viewed as a variant of *Meta-Learning* (Xiang et al., 2025; Duan et al., 2016) or *In-Context Reinforcement Learning* (Nie et al., 2024; Tajwar et al., 2025; Schmied et al., 2025), where the focus is to improve solution quality during test-time with feedback (Qu et al., 2025); but adapted in a novel way to the challenging real-world setting of GPU kernel generation and code optimization.

3 Experimental Setup & Baseline

3.1 Environment and Evaluation

We use KernelBench (Ouyang et al., 2025), a popular benchmark for evaluating LLMs' ability to generate CUDA kernels for deep learning workloads in PyTorch. We chose 180 of the 100 Level 1 problems (basic operators: convolutions, matrix multiplies, loss functions, etc.) and 100 Level 2 problems (sequences of operators with fusion opportunities) as training environments. Since KernelBench does not provide a train–test split, we asked the authors to construct 80 additional tasks following the same methodology (see Appendix). We build the evaluation set by combining our 80 newly created tasks with the 20 remaining original KernelBench tasks, for a total of 100 evaluation tasks. This split ensures the training set covers common CUDA patterns while the test set probes generalisation to new operator combinations.

Each KernelBench task consists in generating a CUDA kernel given a PyTorch reference implementation, which is used to evaluate correctness and speed-up. In our set-up, we evaluate the

model-generated kernels as follows: we verify the output is in the correct format (ensure resultant code is only implemented with inline CUDA) and check for reward hacking (Section 5.2). We then evaluate the kernel for compilation, runtime errors, and correctness. If the implementation is correct, we profile the kernel for its runtime. All kernels are compiled and timed with `torch.compile` on a single node equipped with eight NVIDIA H200 GPUs (1890 MHz). An asynchronous compile–run queue keeps every GPU busy, reducing measurement noise and idle time.

3.2 Basic Kernel Score Design

As we are concerned both with correctness and speed-up, we assign a score S for each kernel evaluation result that effectively balances the correctness–performance trade-off:

$$S \;=\; 0.3 \cdot \mathbf{1}_{\{\text{correct}\}} + \frac{T_{\text{baseline}}}{T_{\text{kernel}}}\,\mathbf{1}_{\{\text{correct}\}}.$$

Correctness is checked against the reference program when tested with randomized inputs; speed-up is computed as the ratio between PyTorch baseline time and kernel runtime. We experimented with various weights of correctness and speed-up, finding this configuration through ablations on models ranging from 7B to 32B.

In addition, we explored rewarding intermediate objectives (successfully compile or execute), yet this caused the model to over-optimize for intermediate steps (e.g. generating kernels that only compile, but aren't necessarily correct). We also experimented with a length penalty on the response, as suggested by Team (2025a), but found that it degrades our model's performance during training.

As a note, we further experiment with more score designs leveraging more fine-grained signals in Section 6.2 and explore their effect through ablations.

3.3 Baseline: Single-Turn GRPO Training

We first establish a single-turn baseline by training `Qwen QwQ-32B` to emit one candidate kernel per prompt without iterative feedback. We apply GRPO (Shao et al., 2024) to train the model. In each training step, we sample 16 responses per task and assign the evaluated score as the reward for each kernel. We compute the GRPO loss according to (Shao et al., 2024), which updates the policy by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}\big[q \sim P(Q),\ \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{\text{old}}}(O|q)\big]$$

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}\mid q,o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}\mid q,o_{i,<t})}\hat{A}_{i,t},\text{clip}\left(\frac{\pi_\theta(o_{i,t}\mid q,o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}\mid q,o_{i,<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t}\right]-\beta D_{KL}\big(\pi_\theta\,\|\,\pi_{\text{ref}}\big)\right\} \tag{1}$$

where $\hat{A}_{i,t} = \dfrac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$, and r_i is the score of a specific kernel.

We note the importance of using a base model with strong enough priors to obtain a non-sparse reward for correctness and speed-up in the beginning of training. For instance, training on `DeepSeek-R1-Distill-Qwen7B` (DeepSeek-AI, 2025) exhibited reward hacking (see Section 5.2) and failed to learn.

Hence, we use a stronger base model, `Qwen QwQ-32B` (Team, 2025c). We perform two gradient steps for a batch (1 on-policy, 1 off-policy) following (Shao et al., 2024). We use `max_response_length` = 16 384. Following (Yu et al., 2025), we apply `Clip-Higher`, decoupling the lower and higher clipping range (0.2 and 0.28 respectively). We sample with `temperature`=0.9 for both training and inference. We set the KL coefficient to 0 to allow the model to deviate freely from the base policy, following (Luo et al., 2025a).

We observe that reward plateaus after 50 steps, likely because single-turn training prevents the model from refining its kernels. Many generated kernels are nearly correct—often a syntax or compilation fix away—but still receive 0 reward, discouraging the model from producing them. Similarly, the correct kernels do not achieve high speed-up, as the model optimizes for correctness rather than attempting a risky approach. We address these limitations through multi-turn training.

4 Multi-Turn GRPO Training

GPU kernel synthesis is a continuous optimization problem: once a kernel is correct, human experts iteratively refine it until performance is met. Single-turn RL only uses the reward of the one generated kernel, causing reward quickly plateaus with kernels with overly conservative optimizations. To enable better sample effiency and discovery of faster kernels, we explicitly incorporate successive turns of code generation, execution, and feedback into each RL training step. Concretely, at each turn the model receives the measured speed-up of its previous attempt and decides how to refine the code. We then proceed to treat every turn as a training sample.

4.1 Algorithm and Setup

In each multi-turn training step:

1. For each task, we sample m parallel trajectories with n refinement turns. To improve sample efficiency, each refinement turn (CoT + response) in a trajectory becomes a single training sample. The response of the model after the CoT consists of a kernel and a CoT summary.
2. We construct the context of a sample by including the history of previous responses, which include generated kernels along with their summarized CoTs, and evaluation feedback.
3. We evaluate the generated kernel and compute its score as shown in Section 3.2. The reward of each turn (CoT + response) is the discounted sum of current and subsequent scores, which we elaborate in Section 4.3.
4. For each task, we normalize the rewards across the mn samples for advantage calculation. Then we compute the GRPO loss over the entire batch.

Formally, this is shown in Algorithm 1.

Algorithm 1 MULTITURNGRPOSTEP$(\mathcal{B}, \pi_\theta, m, n, \gamma)$

Require: batch $\mathcal{B}$ of tasks, current policy π_θ, m parallel trajectories, n refinement turns, discount factor γ

$\mathcal{S} \leftarrow \varnothing$ ▷ buffer of (*context*, *output*, *return*) triples
for all task $\tau \in \mathcal{B}$ **do**
  for $j = 1$ **to** m **do** ▷ sample m trajectories
    $H \leftarrow \{\textsc{Prompt}(\tau)\}$ ▷ history
    for $t = 1$ **to** n **do** ▷ n refinement turns
      $c_t \leftarrow H$ ▷ build context
      $(\mathrm{CoT}, k) \sim \pi_\theta(\cdot \mid c_t)$ ▷ sample reasoning & kernel
      $r_t \leftarrow \textsc{Score}(k)$ ▷ compile, run, compute Eq. (1)
      $\bar{\mathrm{CoT}} \leftarrow \textsc{Summarise}(\mathrm{CoT})$
      $H \leftarrow H \cup \{(\bar{\mathrm{CoT}}, k, r_t)\}$ ▷ append summary, kernel, feedback
    ▷ compute discounted returns for every turn
    for $t = 1$ **to** n **do**
      $R_t \leftarrow \sum_{i=t}^{n} \gamma^{i-t} r_i$
      $\mathcal{S} \leftarrow \mathcal{S} \cup \{(c_t, (\mathrm{CoT}, k), R_t)\}$
  $\textsc{NormalizeRewards}(\mathcal{S} \mid \tau)$
$\textsc{UpdateGRPO}(\pi_\theta, \mathcal{S})$ ▷ one on- and one off-policy gradient step

4.2 Design Choices for Multi-Turn RL

Here we further eloborate our multi-turn algorithm design as shown in Figrue 1 and Algorithm 1.

1. **Sparse rewards & context bloat.** Given the long-horizon and difficult nature of this task, we have very sparse reward. In addition, the raw chains-of-thought (CoTs) for kernel synthesis can exceed 50k-100k tokens in a few turns. We therefore *treat each turn as a separate GRPO sample* to be more sample efficient, converting an n-turn trajectory into n training examples and quadrupling usable data in our $n = 4$ setting. After every turn the

model emits a concise "CoT summary" of the edits it just made; the raw CoT is discarded. The prompt for the next turn thus contains {*Summaries*, kernels, feedback} from all earlier turns, preserving reasoning cues without overflowing the context window.

2. **Rewarding useful but sub-optimal kernels.** Early turns often uncover "half-way" kernels that merely compile, run, or are correct yet still slow. Instead of discarding these attempts, we use them to teach the agent that incremental progress matters: compiling without errors is better than failing to build, and being correct—however suboptimal—is preferable to crashing. Once the agent reliably reaches correctness, it can then focus on performance. Using these seemingly suboptimal samples that eventually lead to good outcome is an idea used in Open-Ended Exploration and Meta-RL, such as Qu et al. (2025).
3. **Guarding against reward hacking.** Rule-based filters reject trivial or degenerate tactics: copying the reference implementation verbatim, delegating expensive work to PyTorch, or printing fabricated timing data. Only kernels that compile, run, and report honest speed-ups contribute to the reward.

Together, per-turn sampling tackles reward sparsity, CoT summarisation keeps contexts tractable, and strict filters maintain reward integrity. These choices are all essential for stable multi-turn training.

4.3 Reward Aggregation and Discounting

We initially explored two naive strategies for multi-turn credit assignment. The greedy approach assigns to each turn its corresponding kernel score, while the outcome-based approach assigns to all turns the best score in the trajectory. The former failed to reward early suboptimal turns that lead to performant kernels later, while the latter ignores individual contributions and is sample inefficient.

Our method balances both approaches by aggregating the future kernels scores with a discount factor. We conduct ablations on the reward formulation. For score aggregation, we can either take the sum $R_t = \sum_{i=t}^{T} \gamma^{i-t} r_i$ or maximum $R_t = \max_{i=t,\ldots,T} \left\{\gamma^{i-t} r_i\right\}$ over future scores. Sum favors generating multiple good kernels, while max prioritizes achieving one high-performing kernel. We evaluate both forms with $\gamma = 0.4$ and $\gamma = 0.8$.

Experiments show that sum with $\gamma = 0.4$ scales best over 8 turns, though max performs better with $\gamma = 0.8$ with fewer turns.

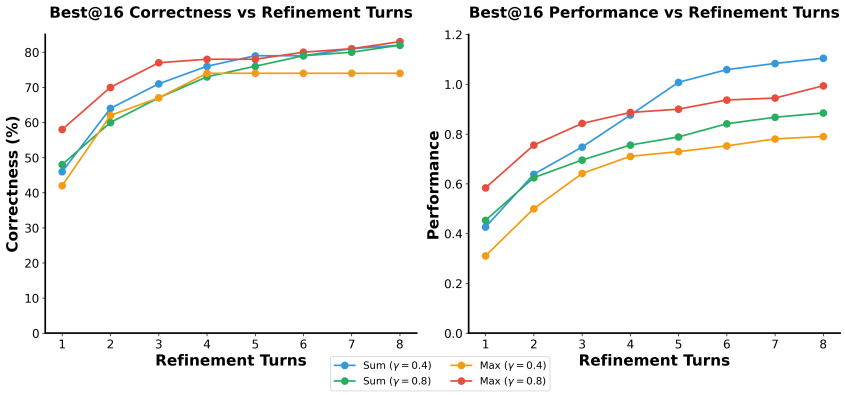

Figure 2: **Sum with $\gamma = 0.4$ is the most effective reward formulation.** Here we compare how models trained with different reward formulations scale with refinement turns.

4.4 Multi-Turn Training Behavior

For our final training run for Bob, we use 16 parallel trajectories and 4 refinement turns per task. Each batch contains 8 tasks. We use the sum reward formulation with discount factor $\gamma = 0.4$. Unlike single-turn training, we observe reward steadily increases; response length behaviors are similar to (Luo et al., 2025b): the response length initially decreases, and then it starts increasing again as the model attempts more sophisticated solutions. Following (Luo et al., 2025b), we extend the max response length from 16K to 22K tokens at step 30.

5 Results and Evaluation

As kernel generation is a challenging task, models are often given extensive test-time compute to tackle it. In our inference setting, we employ multiple parallel trajectories, where each trajectory is made up of several serial turns.

We mark a given trajectory correct if it contains at least one correct kernel. Its performance is the speedup of the fastest kernel (within the trajectory) over the PyTorch Eager reference (speedup of 0x if no kernel is correct). We also consider the $\textbf{fast}_p$ metric, introduced by (Ouyang et al., 2025), which is a binary indicator for whether a trajectory contains a correct kernel with performance of p or more. To aggregate a metric across k parallel trajectories for a given task, we compute: **best@k**, the maximum for that metric across all trajectories; **avg@k**, the average value across trajectories.

5.1 Result on KernelBench Eval Set

We compare Bob against frontier models and the single-turn RL baseline on our aforementioned KernelBench eval set of 100 tasks (Section 3.1), with 16 parallel trajectories, 8 serial refinement turns. As shown in Table 1, Bob achieves a higher performance than its single-turn trained counterpart and other frontier models, demonstrating significant improvement from its base model (`QwQ-32B`). Qualitatively, Bob is able to more effectively implement more aggressive optimizations across several turns (see Appendix for examples and more details).

Model	Correctness		Performance		$\textbf{fast}_1$		$\textbf{fast}_{1.5}$	
	best@16	**avg@16**	**best@16**	**avg@16**	**best@16**	**avg@16**	**best@16**	**avg@16**
Bob (Multi-Turn)	**82%**	**46%**	**1.10x**	**0.40x**	**43%**	15%	**20%**	**6%**
Single-Turn RL	**82**%	45%	0.85x	0.35x	**43**%	**16%**	16%	4%
Qwen QwQ-32B	56%	11%	0.53x	0.08x	23%	3%	10%	1%
OpenAI o4-mini	38%	22%	0.78x	0.27x	21%	7%	13%	**6%**
OpenAI o3-mini	27%	8%	0.30x	0.08x	9%	2%	4%	2%

Table 1: **Bob, trained with our multi-turn RL setup, outperforms other models in terms of correctness and performance.** Here we evaluate models on 100 unseen KernelBench tasks, under a test-time compute setup of 16 parallel trajectories with 8 refinement turns each trajectory.

Multi-turn training lifts median speed-up from **0.85**× to **1.10**×—a 29% gain—while preserving best-of-16 correctness at **82 %**. It also nudges *average* correctness slightly upward (46 % vs. 45%), and the reward curve keeps climbing long after the single-turn baseline plateaus (Fig. 3). These results confirm that iterative credit propagation lets the model take bolder optimizations without sacrificing reliability

5.2 Reward Hacking

In our early experiments with smaller models like `DeepSeek-R1-Distill-Qwen-7B`, we observed frequent reward hacking: the model calls the reference implementation (PyTorch) by directly copying it, wrapping it in try-except statements, or inheriting the reference implementation method. Reward hacking typically emerges when the model capabilities falls short of task difficulty Amodei et al. (2016). In our setting, when model fails to produce correct kernels, the "hacked' kernels are likely the only ones receiving positive reward and get disproportionately reinforced due to advantage normalization. To prevent this, we upgraded our base model to the more capable `QwQ-32B` model as a stronger prior.

However, we observe instances of reward hacking even for stronger models. For Level 2 tasks (targeting kernel fusion), we observe that the model only fuses simple operators (e.g. ReLU, Max), leaving the operator worth optimizing (e.g. convolutions) unfused and unmodified (left in PyTorch). To prevent this, we impose stricter format checks that assign 0 reward to responses with any PyTorch functional operators. We elaborate more on concrete examples of reward hacks in the Appendix.

5.3 Data Distribution

We found it critical to have a balanced difficulty distribution across the dataset, so that on average each batch contains both easier and harder tasks. In one experiment with `DeepSeek-R1-Distill-Qwen-14B` DeepSeek-AI (2025), we trained on a subset of only easy tasks. We observed that the reward quickly plateaus as the model overfits to a single difficulty level. Thus, we address this issue by using a stronger base model `QwQ-32B` and training on both level 1 and 2 of the dataset, which contained tasks with a variety of difficulty and associated optimization techniques.

5.4 Observation on Turn-by-Turn Reward

We take a deeper look at behaviors of model-generated kernels during the multi-turn training process (up to 4 turns per problem). We found Turns 1–2 contribute most to correctness, while Turn 4 account for 70 % of the eventual speed-ups. This reinforces our design choice of using *discount factor* γ=0.4: it still credits early correctness but leaves enough future value on the table to encourage exploration.

6 Discussion

6.1 Increasing Number of Turns During Training

As we found multi-turn (4-turn) RL worked gave us significant gain during training and downstream eval, we want to understand *how many turns is enough?* As shown in Figure 3, we compare training with 1-, 2-, and 4-turn budgets. We found even With just two turns the policy learn quickly, reaching an EMA reward of ≈ 0.42 after only ~ 25 gradient steps, but the does collapse eventually! This shows the critical benefits of iterating from feedback, yet more steps are still needed to avoid model learning pre-mature optimizations (that lead to eventual collapse in the 2-turn case).

(a) 1-turn Baseline (b) 2-turn Training Reward (c) 4-turn Training Reward

Figure 3: Train Reward (EMA) over optimization steps for different turn budgets. Four turns performs the best over time, despite being outpaced by two-turn for the first 25 steps (which collapsed later).

For our 4-turn training run, in its first 25 steps, the reward curve trails the 2-turn training run, reflecting two possible intertwined effects:

(a) *Larger search space.* Each additional edit multiplies the branching factor, so the policy must sift through a wider array of low-value trajectories before discovering consistently good ones.

(b) *Slower consistent gains.* After the most impactful optimiations are applied in the first one or two edits, later turns often perform fine-grained tweaks that add little—or occasionally reverse earlier improvements—slowing visible learning progress in the early stages, and it takes more time for the model to learn when to implement the correct optimizations, but this exploration seems to pay off as four-turn does not experience the same collapse as two-turn training.

Around step 35 the curve crosses over: the extra degrees of freedom begin to outweigh their early cost. Training with 4 turns allow the policy to chain together *complementary* optimizations that are impossible to express within two edits. The reward continues to climb above the 2-turn ceiling (we measure an EMA of ≈ 0.47 after 40 steps).

6.2 Reward Design and Ablation

Our reward (for both single- and multi-turn training) so far uses or aggregates the score design in Section 3.2, which simply just considered correctness and speedup. As we observed frequent runtime errors and overly conservative solutions, we explore designing alternative reward variants that shift emphasis between *correctness, stability, and performance*. We therefore experiment with fine-grained awards that assign small bonuses to partial progress (BUILD or EXEC success) and let us bias the policy toward either stability or raw performance without changing the optimization algorithm itself.

6.2.1 Reward variants

We augment the score related to kernel evaluation with the following fine-grained signal.

$$B = \mathbf{1}_{\text{built}} \in \{0, 1\}, \tag{2}$$

$$E = \mathbf{1}_{\text{execute}} \in \{0, 1\}, \tag{3}$$

$$C = \mathbf{1}_{\text{correct}} \in \{0, 1\}, \tag{4}$$

$$F = \frac{t_{\text{baseline}}}{t_{\text{new}}}, \tag{5}$$

$$P = \tanh\big(\beta_F \, \ln(1 + F)\big), \tag{6}$$

$$L_s = \exp\Big(-\tfrac{(L - L_{\text{target}})^2}{2\sigma_L^2}\Big), \tag{7}$$

$$S_{\text{new}} = \begin{cases} 0, & C = 0, \\ \alpha_B \, B + \alpha_E \, E + \alpha_C \, C + \alpha_P \, P + \alpha_L \, L_s, & C = 1, \end{cases} \tag{8}$$

The binary indicators B, E, C provide low-cost feedback for build and execution progress, P smoothly rewards speed-ups with diminishing returns, and L_s discourages bloated responses. Setting the reward to zero when $C = 0$ blocks reward-hacking trajectories that never reach correctness. In Table 2, we define a few strategies with different coefficients $\alpha_B, \alpha_E, \alpha_C, \alpha_P, \alpha_L$.

Strategy	B	E	C	P	L	Qualitative behaviour
`BALANCED`	0.05	0.05	0.30	0.50	0.10	Smooth gradients; steady but slower learning
`STABILITY-CENTRIC`	0.10	0.10	0.60	0.15	0.05	Rapidly removes runtime errors; explores less aggressively
`PERFORMANCE-CENTRIC`	0.05	0.05	0.20	0.65	0.05	Takes risky edits to chase speed-ups once correctness is reached

Table 2: **Reward-shaping strategies.** B = Build, E = Execute, C = Correctness, P = Performance, L = Length penalty. All weights sum to 1; larger values place more emphasis on the corresponding signal.

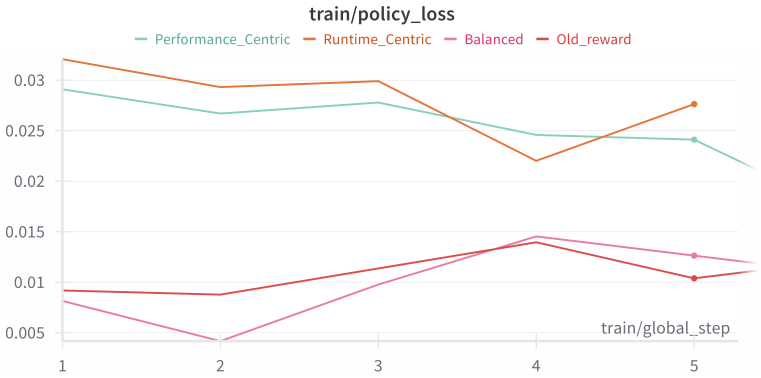

Figure 4: **Effect of different reward design on policy loss.** We plot `train/policy_loss` for 4 reward-shaping variants during multi-turn (4-turn) training. Although policy loss is not an evaluation metric, its magnitude and smoothness reflect how easy each reward signal is to optimize.

Due to the costly nature of full multi-turn runs, we tracked policy loss for only the first 5 GRPO updates. During this window, the Balanced signal descends most gently yet reaches the lowest loss, reflecting a smooth but slow optimization surface. Both Performance-Heavy and Runtime-Robust start from larger loss values; their steeper drops suggest they learn more aggressively once rewards appear. Notably, the Runtime-Robust curve exhibits a brief spike after step 3, coinciding with several observational time-outs in the kernel logs. We see this as an early hint that rewarding runtime alone can inject variance before correctness is established. Given their faster initial convergence, we expect the two aggressive variants would overtake Balanced if training were extended beyond five updates.

7 Conclusion

7.1 Summary

We designed a multi-turn RL training recipe that addresses challenges when applied to the real-world task of GPU kernel generation: specifically, effective context management and credit attribution across every turn to enable better sample efficiency. We also implemented mechanisms to prevent reward hacking.

We present Bob, a model trained with multi-turn RL to generate CUDA kernels, on KernelBench Level 1 and 2 tasks. Evaluated on an unseen KernelBench evaluation set, Bob outperforms its single-turn RL counterpart and frontier models, demonstrating that our training recipe enables the model to learn more effective refinement strategies.

As multi-turn RL gave us significant gain, we want to understand the effect of increasing turns during training rollout. Increasing to 2 turns enable quick reward gain immediately, yet eventually model collapse as it does premature optimizations and does not learn more advanced and subtle optimizations that 4-turn can learn. We also experiment with a variety of fine-grained reward signal in hope to steer policy with more partial credits, but we could not fully explore these tradeoffs due to our compute constraint.

7.2 Limitations

Despite promising results, our approach has several limitations:

1. **Model stability and training budget.** The base model (`QwQ-32B` Team (2025c)) is already heavily post-trained, so prolonged RL fine-tuning can destabilise it Team et al. (2025). To remain in a safe regime, we cap training at 80 gradient steps, which also limits exhaustive hyper-parameter sweeps (e.g. additional turn budgets or reward schedules).

2. **Compute-driven ablation limits.** Restricted GPU hours prevent broader experiments—such as systematically varying the number of turns during multi-turn RL beyond four turns—so we leave those studies to future work.
3. **Hardware and shape specificity.** KernelBench tasks come with fixed input shapes, and all reported speedups were obtained on NVIDIA H200 GPUs. Performance may differ for other tensor dimensions or hardware generations.
4. **Memory-hierarchy coverage.** Current benchmarks fit comfortably within shared memory; kernels that exercise deeper or more irregular memory hierarchies remain challenging.

7.3 Future Work

We outline several directions for extending our method. Incorporating a learned value network and using Proximal Policy Optimization Schulman et al. (2017) might improve the baseline estimation during training. At training and test-time, we could implement more sophisticated search methods such as beam search or Monte-Carlo Tree Search Silver et al. (2017). Inspired by recent works Sareen et al. (2025), we could also leverage the value network as a verifier for search at test-time.

Our multi-turn RL process demonstrates success in the real-world engineering task of GPU kernel generation. However, we designed this recipe in a flexible manner, potentially applicable to a wider range of tasks that feature verifiable rewards and execution feedback across a trajectory. We believe explicitly training models to reason about complex tasks over multiple turns to be a key step towards enabling autonomous AI systems.

8 Team Contributions

- **Group Member 1: Simon** Set up the whole training pipeline and got results for baseline 1-turn and 4-turn RL training runs.
- **Group Member 2:Shafin Khan** led the 2-turn run and worked on fine-grained reward ablation.
- **Group Member 3: William Li** led the fine-grained reward design and also worked on the 2-turn training run.

Note: Bob is part of a research project that Simon (PhD student) actively works on. Part of this project is currently under submission for conferences. Shafin and Will both substantianly contribute to crucial abalations as outline in Section 6.

Changes from Proposal Due to the limited amount of GPU we have, we deliberately avoided any experiments that required full end-to-end training runs. Instead, we redirected our effort toward reward-function ablations and inspected only the first few optimization steps. This strategy allows us to capture early learning signals and iterate on reward design without incurring the high computational cost of full-length training schedules.

For experiments that require full training runs like vary the number of turns during training roll-outs, we could not sweep too many configurations (only 1,2,4-turn) as each run takes 3-4 days to run on a 8xH200 setup.

References

Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete Problems in AI Safety. arXiv:1606.06565 [cs.AI] https://arxiv.org/abs/1606.06565

Shiyi Cao, Sumanth Hegde, Dacheng Li, Tyler Griggs, Shu Liu, Eric Tang, Jiayi Pan, Xingyao Wang, Akshay Malik, Graham Neubig, Kourosh Hakhamaneshi, Richard Liaw, Philipp Moritz, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. SkyRL-v0: Train Real-World Long-Horizon Agents via Reinforcement Learning.

Terry Chen, Bing Xu, and Kirthi Devleker. 2025. Automating GPU Kernel Generation with DeepSeek-R1 and Inference-Time Scaling. https://developer.nvidia.com/blog/automating-gpu-kernel-generation-with-deepseek-r1-and-inference-time-scaling/. Accessed: 2025-05-15.

Sana Damani, Siva Kumar Sastry Hari, Mark Stephenson, and Christos Kozyrakis. 2024. WarpDrive: An Agentic Workflow for Ninja GPU Transformations. In *Proceedings of the Machine Learning for Systems Workshop at NeurIPS 2024*. https://mlforsystems.org/assets/papers/neurips2024/paper32.pdf Accessed: 2025-05-15.

Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691 [cs.LG] https://arxiv.org/abs/2307.08691

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv:2205.14135 [cs.LG] https://arxiv.org/abs/2205.14135

DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.CL] https://arxiv.org/abs/2501.12948

Tulsee Doshi. 2025. Gemini 2.5: Our most intelligent models are getting even better. https://blog.google/technology/google-deepmind/google-gemini-updates-io-2025/. Accessed: 2025-05-21.

Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. 2016. RL^2: Fast Reinforcement Learning via Slow Reinforcement Learning. arXiv:1611.02779 [cs.AI] https://arxiv.org/abs/1611.02779

Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Quentin Carbonneaux, Taco Cohen, and Gabriel Synnaeve. 2025. RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning. arXiv:2410.02089 [cs.CL] https://arxiv.org/abs/2410.02089

Anna Goldie, Azalia Mirhoseini, Hao Zhou, Irene Cai, and Christopher D. Manning. 2025. Synthetic Data Generation & Multi-Step RL for Reasoning & Tool Use. arXiv:2504.04736 [cs.AI] https://arxiv.org/abs/2504.04736

Google DeepMind. 2025. AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms. https://deepmind.google/discover/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/ Accessed: 2025-05-15.

Jian Hu, Xibin Wu, Zilin Zhu, Xianyu, Weixun Wang, Dehao Zhang, and Yu Cao. 2024. OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework. arXiv:2405.11143 [cs.AI] https://arxiv.org/abs/2405.11143

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770 [cs.CL] https://arxiv.org/abs/2310.06770

Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, Dzmitry Bahdanau, Leandro von Werra, and Harm de Vries. 2022. The Stack: 3 TB of permissively licensed source code. arXiv:2211.15533 [cs.CL] https://arxiv.org/abs/2211.15533

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. 2025. Tulu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124 [cs.CL] https://arxiv.org/abs/2411.15124

Robert Tjarko Lange, Aaditya Prasad, Qi Sun, Maxence Faldor, Yujin Tang, and David Ha. 2025. The AI CUDA Engineer: Agentic CUDA Kernel Discovery, Optimization and Composition. https://pub.sakana.ai/static/paper.pdf Accessed: 2025-05-15.

Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. 2025. TritonBench: Benchmarking Large Language Model Capabilities for Generating Triton Operators. arXiv:2502.14752 [cs.CL] https://arxiv.org/abs/2502.14752

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2023. StarCoder: may the source be with you! arXiv:2305.06161 [cs.CL] https://arxiv.org/abs/2305.06161

Jiate Liu, Yiqin Zhu, Kaiwen Xiao, Qiang Fu, Xiao Han, Wei Yang, and Deheng Ye. 2023. RLTF: Reinforcement Learning from Unit Test Feedback. arXiv:2307.04349 [cs.AI] https://arxiv.org/abs/2307.04349

Michael Luo, Sijun Tan, Roy Huang, Ameen Patel, Alpay Ariyak, Qingyang Wu, Xiaoxiang Shi, Rachel Xin, Colin Cai, Maurice Weber, Ce Zhang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. 2025a. DeepCoder: A Fully Open-Source 14B Coder at O3-mini Level. https://pretty-radio-b75.notion.site/DeepCoder-A-Fully-Open-Source-14B-Coder-at-O3-mini-Level-1cf81902c14680b3bee5eb349a512a51. Notion Blog.

Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. 2025b. DeepScaleR: Surpassing O1-Preview with a 1.5B Model by Scaling RL. https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a Notion Blog.

METR. 2025. Measuring Automated Kernel Engineering. https://metr.org/blog/2025-02-14-measuring-automated-kernel-engineering/ Accessed: 2025-05-15.

John Nickolls, Ian Buck, Michael Garland, and Kevin Skadron. 2008. Scalable parallel programming with CUDA. In *ACM SIGGRAPH 2008 Classes* (Los Angeles, California) *(SIGGRAPH '08)*. Association for Computing Machinery, New York, NY, USA, Article 16, 14 pages. https://doi.org/10.1145/1401132.1401152

Allen Nie, Yi Su, Bo Chang, Jonathan N. Lee, Ed H. Chi, Quoc V. Le, and Minmin Chen. 2024. EVOLvE: Evaluating and Optimizing LLMs For Exploration. arXiv:2410.06238 [cs.LG] https://arxiv.org/abs/2410.06238

NVIDIA. 2025. GPU MODE at NVIDIA GTC 2025. https://www.youtube.com/watch?v=mdDVkBeFy9A Accessed: 2025-05-15.

NVIDIA Corporation. 2025. CUTLASS: CUDA Templates for Linear Algebra Subroutines. https://github.com/NVIDIA/cutlass Accessed: 2025-05-15.

Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. 2025. KernelBench: Can LLMs Write Efficient GPU Kernels? arXiv:2502.10517 [cs.LG] https://arxiv.org/abs/2502.10517

Yuxiao Qu, Matthew Y. R. Yang, Amrith Setlur, Lewis Tunstall, Edward Emanuel Beeching, Ruslan Salakhutdinov, and Aviral Kumar. 2025. Optimizing Test-Time Compute via Meta Reinforcement Fine-Tuning. arXiv:2503.07572 [cs.LG] https://arxiv.org/abs/2503.07572

Kusha Sareen, Morgane M Moss, Alessandro Sordoni, Rishabh Agarwal, and Arian Hosseini. 2025. Putting the Value Back in RL: Better Test-Time Scaling by Unifying LLM Reasoners With Verifiers. arXiv:2505.04842 [cs.LG] https://arxiv.org/abs/2505.04842

Thomas Schmied, Jörg Bornschein, Jordi Grau-Moya, Markus Wulfmeier, and Razvan Pascanu. 2025. LLMs are Greedy Agents: Effects of RL Fine-tuning on Decision-Making Abilities. arXiv:2504.16078 [cs.LG] https://arxiv.org/abs/2504.16078

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347 [cs.LG] https://arxiv.org/abs/1707.06347

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300 [cs.CL] https://arxiv.org/abs/2402.03300

Alexander Shypula, Aman Madaan, Yimeng Zeng, Uri Alon, Jacob Gardner, Milad Hashemi, Graham Neubig, Parthasarathy Ranganathan, Osbert Bastani, and Amir Yazdanbakhsh. 2024. Learning Performance-Improving Code Edits. arXiv:2302.07867 [cs.SE] https://arxiv.org/abs/2302.07867

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. 2017. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. arXiv:1712.01815 [cs.AI] https://arxiv.org/abs/1712.01815

Benjamin F. Spector, Simran Arora, Aaryan Singhal, Daniel Y. Fu, and Christopher Ré. 2024. ThunderKittens: Simple, Fast, and Adorable AI Kernels. arXiv:2410.20399 [cs.LG] https://arxiv.org/abs/2410.20399

Fahim Tajwar, Yiding Jiang, Abitha Thankaraj, Sumaita Sadia Rahman, J Zico Kolter, Jeff Schneider, and Ruslan Salakhutdinov. 2025. Training a Generally Curious Agent. arXiv:2502.17543 [cs.LG] https://arxiv.org/abs/2502.17543

Kimi Team. 2025a. Kimi k1.5: Scaling Reinforcement Learning with LLMs. arXiv:2501.12599 [cs.AI] https://arxiv.org/abs/2501.12599

NovaSky Team. 2025b. Sky-T1: Train your own O1 preview model within $450. https://novasky-ai.github.io/posts/sky-t1. Accessed: 2025-01-09.

Prime Intellect Team, Sami Jaghouar, Justus Mattern, Jack Min Ong, Jannik Straube, Manveer Basra, Aaron Pazdera, Kushal Thaman, Matthew Di Ferrante, Felix Gabriel, Fares Obeid, Kemal Erdem, Michael Keiblinger, and Johannes Hagemann. 2025. INTELLECT-2: A Reasoning Model Trained Through Globally Decentralized Reinforcement Learning. arXiv:2505.07291 [cs.LG] https://arxiv.org/abs/2505.07291

Qwen Team. 2025c. QwQ-32B: Embracing the Power of Reinforcement Learning. https://qwenlm.github.io/blog/qwq-32b/

Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (Phoenix, AZ, USA) *(MAPL 2019)*. Association for Computing Machinery, New York, NY, USA, 10–19. https://doi.org/10.1145/3315508.3329973

Siddhant Waghjale, Vishruth Veerendranath, Zora Zhiruo Wang, and Daniel Fried. 2024. ECCO: Can We Improve Model-Generated Code Efficiency Without Sacrificing Functional Correctness? arXiv:2407.14044 [cs.CL] https://arxiv.org/abs/2407.14044

Guanhua Wang, Heyang Qin, Sam Ade Jacobs, Connor Holmes, Samyam Rajbhandari, Olatunji Ruwase, Feng Yan, Lei Yang, and Yuxiong He. 2023. ZeRO++: Extremely Efficient Collective Communication for Giant Model Training. arXiv:2306.10209 [cs.DC] https://arxiv.org/abs/2306.10209

Yiping Wang, Qing Yang, Zhiyuan Zeng, Liliang Ren, Lucas Liu, Baolin Peng, Hao Cheng, Xuehai He, Kuan Wang, Jianfeng Gao, Weizhu Chen, Shuohang Wang, Simon Shaolei Du, and Yelong Shen. 2025b. Reinforcement Learning for Reasoning in Large Language Models with One Training Example. arXiv:2504.20571 [cs.LG] https://arxiv.org/abs/2504.20571

Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, Monica Lam, Yiping Lu, Kyunghyun Cho, Jiajun Wu, Li Fei-Fei, Lijuan Wang, Yejin Choi, and Manling Li. 2025a. RAGEN: Understanding Self-Evolution in LLM Agents via Multi-Turn Reinforcement Learning. arXiv:2504.20073 [cs.LG] https://arxiv.org/abs/2504.20073

Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. 2025. SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution. arXiv:2502.18449 [cs.SE] https://arxiv.org/abs/2502.18449

Violet Xiang, Charlie Snell, Kanishk Gandhi, Alon Albalak, Anikait Singh, Chase Blagden, Duy Phung, Rafael Rafailov, Nathan Lile, Dakota Mahan, Louis Castricato, Jan-Philipp Franken, Nick Haber, and Chelsea Finn. 2025. Towards System 2 Reasoning in LLMs: Learning How to Think With Meta Chain-of-Thought. arXiv:2501.04682 [cs.AI] https://arxiv.org/abs/2501.04682

Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. *arXiv preprint arXiv:2501.01005* (2025). https://arxiv.org/abs/2501.01005

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Weinan Dai, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. 2025. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. arXiv:2503.14476 [cs.LG] https://arxiv.org/abs/2503.14476

Chenggang Zhao, Liang Zhao, Jiashi Li, and Zhean Xu. 2025. DeepGEMM: clean and efficient FP8 GEMM kernels with fine-grained scaling. https://github.com/deepseek-ai/DeepGEMM.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. 2022. MiniF2F: a cross-system benchmark for formal Olympiad-level mathematics. arXiv:2109.00110 [cs.AI] https://arxiv.org/abs/2109.00110

Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine, and Aviral Kumar. 2024. ArCHer: Training Language Model Agents via Hierarchical Multi-Turn RL. arXiv:2402.19446 [cs.LG] https://arxiv.org/abs/2402.19446

Richard Zhuang*, Trung Vu*, Alex Dimakis, and Maheswaran Sathiamoorthy. 2025. Improving Multi-Turn Tool Use with Reinforcement Learning. https://www.bespokelabs.ai/blog/improving-multi-turn-tool-use-with-reinforcement-learning. Accessed: 2025-04-17.

A KernelBench Modifications

We use KernelBench Ouyang et al. (2025) as our training environments. KernelBench is a popular benchmark for evaluating LLMs' ability to generate performant CUDA kernels for deep learning workloads in PyTorch. Each KernelBench task consists in generating a CUDA kernel given a PyTorch reference implementation, which is used to evaluate correctness and speedup.

A.1 Task Improvements

We identify several limitations in the original KernelBench and introduce targeted modifications to address them. These changes are crucial to mitigate reward hacking, as shown in Section 5.2.

- We sand-boxed the kernel evaluation process so that fatal errors, such as `CUDA illegal memory accesses`, do not crash the RL training process.
- A significant issue we noted in KernelBench was that for many tasks, the input tensors used to measure performance are quite small. This causes kernel launch overhead to take up a significant portion of the runtime. To address this, we enlarged the tensor dimensions of the affected tasks.
- A sneakier bug in the KernelBench's evaluation harness caused the tested kernel to recycle the output tensor from the reference implementation (which was run immediately before) as its own tensor output. As a result of this, a kernel that only computes (correctly) a portion of the output tensor would still pass the correctness check. We address this by running the tested kernel first and only after the reference implementation, thus avoiding this hack.

In the end, we chose a total of 180 tasks as training environments, with 90 of the 100 Level 1 problems and 90 Level 2 problems (sequences of operators with fusion opportunities).

A.2 Construction of Additional Evaluation Set

Since KernelBench does not provide a train-test split, we asked the authors to construct 80 additional tasks following the same methodology that KernelBench was constructed.

KernelBench Level 2 is constructed by composing a subset of PyTorch operators as sequences of operators. Specifically, the PyTorch operators are categorized as:

- **Main operators:** `Conv2d`, `Matmul`, `Gemm`, `BMM`, `Conv3d`, `ConvTranspose2d`, `ConvTranspose3d`.
- **Activations:** `ReLU`, `Sigmoid`, `Tanh`, `LeakyReLU`, `GELU`, `Swish`, `Softmax`, `Mish`, `Hardtanh`, `HardSwish`.
- **Element-wise operators:** `Add`, `Multiply`, `Subtract`, `Divide`, `Clamp`, `Scale`, `ResidualAdd`.
- **Normalizations:** `BatchNorm`, `LayerNorm`, `InstanceNorm`, `GroupNorm`.
- **Pooling:** `MaxPool`, `AvgPool`, `GlobalAvgPool`.
- **Bias:** `BiasAdd`.
- **Reductions:** `Sum`, `Mean`, `Max`, `Min`, `LogSumExp`.
- **Others:** `ResidualAdd`, `Scaling`.

To construct the additional eval set (unseen from train set), following the methodology from original KernelBench task construction:

1. We sample from the available operators listed above: 1 main operator (computationally expensive), and 2-5 other operators.
2. We ask a language model, namely Gemini 2.5-Flash Doshi (2025), to generate a PyTorch program that creates a kernel by combining these operators. We also ask it to generate sample tensor sizes for the task.
3. We ensure this PyTorch program can be executed and has a runtime on NVIDIA H200 > 0.1ms, to avoid the runtime being dominated by kernel launch (CPU) overhead.

4. We make sure this PyTorch program (with the same sequence of operators) is not present in existing KernelBench Level 1 and 2 programs.

We manually inspected all new task programs to ensure their validity. We build the evaluation set by combining our 80 newly created tasks with the 20 remaining original KernelBench tasks, for a total of 100 unseen evaluation tasks.

B Additional Details on Multi-Turn RL

Here we elaborate on design choices for our RL Training as described in Section 3.3 and Section 4.1, along with some ablation results.

B.1 Motivation for Turn-wise Reward

In our multi-turn RL training setup, within each training step we have a trajectory with n refinement turns. A possible approach would be to compute the reward based on the kernel at the last turn, similar to what is used in RLEF Gehring et al. (2025). However, for the GPU kernel optimization setting, using just the last kernel might not be optimal at times: for example, as shown earlier in Figure 1, kernel 3 is correct but kernel 4 is incorrect as the model attempts more aggressive optimizations.

In this setting, computing reward based on the best kernel among the trajectory instead (max speedup) is a more natural choice. However, using only the max kernel score forces us to discard all turns in a trajectory after the max turn, possibly wasting a significant amount of inference rollouts: In the previous example, we would have to completely discard the reasoning trace, code, and evaluation for kernel 4. Thus, we arrived at our approach in Section 4.3, which uses a discounted look-ahead max or sum, enabling more sample-efficient training.

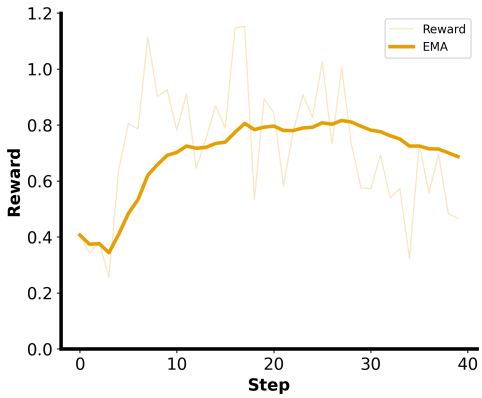

Figure 5: Training reward with correctness weighting of 1, performance / speedup weighting of 1. Concretely, $S = \mathbf{1}_{\{\text{correct}\}} + \frac{T_{\text{baseline}}}{T_{\text{kernel}}} \cdot \mathbf{1}_{\{\text{correct}\}}$.

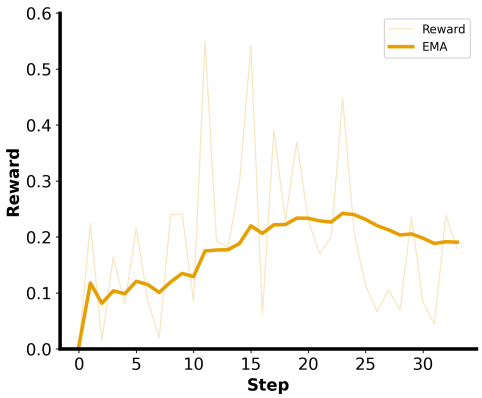

Figure 6: Training reward with no correctness weighting, performance / speedup weighting of 1. (speedup is 0 if kernel is incorrect). Concretely, $S = \mathbf{1}_{\{\text{correct}\}} \cdot \frac{T_{\text{baseline}}}{T_{\text{kernel}}}$.

B.2 Weighting for Score

In Section 3.2, we explain our score design, which assigns a scalar value (score S) based on a kernel's correctness and speedup. We explore score design and how to balance the correctness-performance trade-off, after series of small-scale ablations on `QwQ-32B` Team (2025c).

We decided on a weighting of `0.3` on correctness and using speedup for performance (raw speedup itself, no weighting), which is $S = 0.3 \cdot \mathbf{1}_{\{\text{correct}\}} + \mathbf{1}_{\{\text{correct}\}} \cdot \frac{T_{\text{baseline}}}{T_{\text{kernel}}}$.

Here we present some ablation studies we ran with different weighting configurations for score design, particularly focusing on adjusting the weighing for correctness, in the context of single-turn RL (GRPO) training (as shown in Section 3.3). As show an example in Figure 5, where we set

the weighting to 1.0 for correctness, the reward plateaus and eventually decreased; concretely, we observed that the model over-optimizes for generating correct kernels and does not explore speedup as much, causing the reward to plateau during training. In another experiment in Figure 6, we set the weighting to 0 for correctness, only rewarding the model for generating performant (and correct) kernels. We again observed the reward plateau. Thus, we hypothesize that it is still important to reward the model for correct kernels, as long as the correctness reward is not too significant, balancing the correctness-performance tradeoff.

B.3 Number of Trajectories during Training

We vary the number of parallel trajectories during Multi-Turn RL training (Section 4.1), using 64 parallel trajectories instead of 16 for each task during each training step. We note that best@16 correctness slightly increases, but the overall performance does not show significant improvements. Due to the high-compute requirements of doing more generations during training, we chose to train with 16 parallel trajectories.

B.4 Length Penalty

We explore incorporating response length as a part of the reward design to incentivize the model to use its reasoning tokens more efficiently. We attempted a run using the length penalty from Kimi Team (2025a) on `DeepSeek-R1-Distill-Qwen-14B`. As shown in Figures 7 and 8, we found that the response length of the responses collapses, with the model no longer outputting CoT after 10 training steps, suggesting that the addition of a length penalty is counterproductive for our setting.

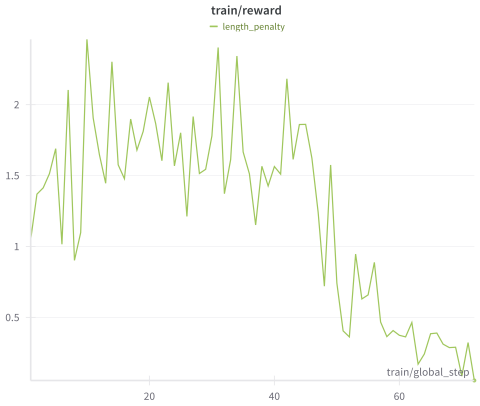

Figure 7: Training Reward collapses when including length penalty as part of reward

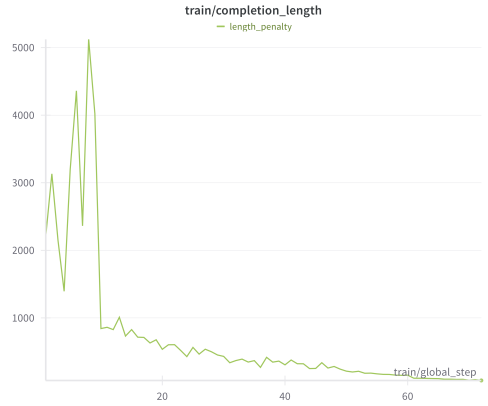

Figure 8: Response length of generations collapses when including length penalty as part of reward.

C RL Infrastructure

Although a few open-source RL frameworks existed when we began this study, it is still difficult to support training in a kernel evaluation environment and including multiple turns within one training step. We built our training framework on top of the OpenRLHF Hu et al. (2024) framework.

We use vLLM Kwon et al. (2023) for inference and DeepSpeed Zero-3 Wang et al. (2023) for offloading optimizer states.

Each of the 8 GPUs handles the kernel generation and evaluation for one task. After the response generation finishes, each GPU offloads its vLLM engine to CPU memory and evaluates the kernels it generated. We run the evaluation and calculate reward and evaluation info. Each GPU then wakes up its corresponding vLLM engine and regenerates kernels.

Each full RL training run took multiple days due to the limited compute we have. Hence to iterate quickly and compare across configurations, we train up to 40-50 global steps (80-100 gradient steps).

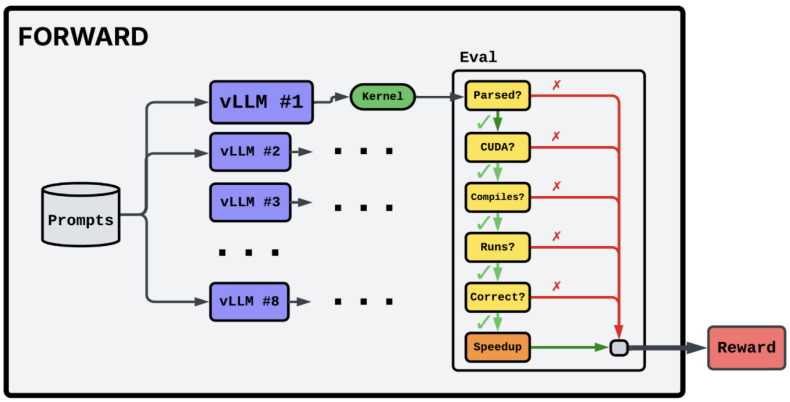

Figure 9: Overview of our RL Training infrastructure.

D Inference Setup

Our prompt is similar to the prompt used in KernelBench Ouyang et al. (2025). We use this during training and test-time inference. In the first refinement turn, we add an example of the inline CUDA format to the prompt but remove it afterwards.

Below we show how we construct the context in the simplest case (of one turn, or the base prompt). In the context, we present model the KernelBench task, instructions, and a simple 1-shot example of a CUDA add kernel (to inform model the desired format for response):

```
You are given the following architecture:
import torch
import torch.nn as nn

class Model(nn.Module):
    """
    Simple model that performs Layer Normalization.
    """
    def __init__(self, normalized_shape: tuple):
        """
        Initializes the LayerNorm layer.

        Args:
            normalized_shape (tuple): Shape of the input tensor to be normalized.
        """
        super(Model, self).__init__()
        self.ln = nn.LayerNorm(normalized_shape=normalized_shape)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Applies Layer Normalization to the input tensor.

        Args:
            x (torch.Tensor): Input tensor of shape (*, normalized_shape).

        Returns:
            torch.Tensor: Output tensor with Layer Normalization applied, same
    shape as input.
        """
        return self.ln(x)

Replace pytorch operators in the given architecture with raw CUDA kernels,
    optimizing for performance on NVIDIA H100 (e.g. shared memory, kernel fusion,
    warp primitives, vectorization,...). Use torch.utils.cpp_extension.load_inline
    and name your optimized output architecture ModelNew. You are not allowed to
```

```
    use torch.nn (except for Parameter, containers, and init). The input and
    output have to be on CUDA device. Your answer must be the complete new
    architecture (no testing code, no other code): it will be evaluated and you
    will be given feedback on its correctness and speedup so you can keep
    iterating, trying to maximize the speedup. After your answer, summarize your
    changes in a few sentences.Here is an example:

import torch.nn as nn
from torch.utils.cpp_extension import load_inline

# Define the custom CUDA kernel for element-wise addition
elementwise_add_source = """
#include <torch/extension.h>
#include <cuda_runtime.h>

__global__ void elementwise_add_kernel(const float* a, const float* b, float* out,
    int size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < size) {
        out[idx] = a[idx] + b[idx];
    }
}

torch::Tensor elementwise_add_cuda(torch::Tensor a, torch::Tensor b) {
    auto size = a.numel();
    auto out = torch::zeros_like(a);

    const int block_size = 256;
    const int num_blocks = (size + block_size - 1) / block_size;

    elementwise_add_kernel<<<num_blocks, block_size>>>(a.data_ptr<float>(),
    b.data_ptr<float>(), out.data_ptr<float>(), size);

    return out;
}
"""

elementwise_add_cpp_source = (
    "torch::Tensor elementwise_add_cuda(torch::Tensor a, torch::Tensor b);"
)

# Compile the inline CUDA code for element-wise addition
elementwise_add = load_inline(
    name="elementwise_add",
    cpp_sources=elementwise_add_cpp_source,
    cuda_sources=elementwise_add_source,
    functions=["elementwise_add_cuda"],
    verbose=True,
    extra_cflags=[""],
    extra_ldflags=[""],
)


class ModelNew(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.elementwise_add = elementwise_add

    def forward(self, a, b):
        return self.elementwise_add.elementwise_add_cuda(a, b)
```

For our multi-turn RL training (Section 4.1) and inference (Section 5), we provide model with the kernels, CoTs (summarized), and evaluation results of all previous turns in chronological order. We truncate the turns that do not fit inside the context window, starting from the earliest ones.

```
<Base prompt containing pytorch architecture and instruction>

Here are your previous attempts:

< for each (i) previously generated kernel >
    <Previously generated kernel G[i]>

    <Summary of CoT[i]>

    <if parsing error>

        Your previous answer failed to be parsed due to not adhering to the desired
    formatting. Here is the error message: <error_message>

    <elif compilation error>

        Your previous answer failed to compile. Here is the error message:
    <error_message>

    <elif run error>

        Your previous answer compiled successfully but had runtime errors. Here is
    the error message: <error_message>

    <elif correctness error>

        Your previous answer was incorrect. Here is the error message:
    <error_message>

    <elif correct>

        Your previous answer was correct but can be made faster. Here is the
    speedup you achieved relative to the baseline: <speedup>

Restart your reasoning process and generate new, complete code.
```

E Reward Hacking

Here we present excerpts from generated kernels that show signs of reward hacking, previously mentioned in Section 5.2.

In the following example, the model simply copies the PyTorch reference implementation, thus getting rewarded for generating a correct answer with 1.0x speedup. To prevent this, we modify our kernel evaluation environment so that it checks each generated kernel if it contains instances of `torch.nn` or `torch.nn.functional`. We assign a reward of 0 to those.

```
class ModelReLU(Module):
    ...
    def forward(self, x):
        relu = torch.nn.ReLU()
        return relu(x)
```

Similarly, the model wraps an incorrect implementation of the CUDA kernel in a try-except statement and invokes the PyTorch implementation functions as a fallback. To prevent this, we assign a reward of 0 to kernels that contain try or except.

```
class ModelReLU(Module):
    ...
    def forward(self, x):
        try:
            ...  \# CUDA implementation
        except Exception as e:
            print("Custom ReLU kernel failed to compile. Using default ReLU
    instead.")
```

```
            relu = torch.nn.ReLU()
            return relu(x)
```

Here the model inherits from the reference implementation, bypassing the need for a CUDA implementation. To prevent this, we assign a reward of 0 to kernels that contain pass.

```
class ModelReLU(Module):
    ...
    def forward(self, x):
        ...   \# CUDA implementation

class ModelReLU(Model):  \# Model is the the reference implementation
    pass
```

F Kernel Highlight

As an example, we look at one the task form our evaluation set, `Conv3D_GELU_Tanh_Clamp`:

```
import torch
import torch.nn as nn

class Model(nn.Module):
    """
    Model using Conv3d, GELU, Clamp, Tanh
    """
    def __init__(self, in_channels, out_channels, kernel_size, clamp_min,
    clamp_max):
        super(Model, self).__init__()
        self.conv = nn.Conv3d(in_channels, out_channels, kernel_size)
        self.clamp_min = clamp_min
        self.clamp_max = clamp_max

    def forward(self, x):
        """
        Forward pass of the model.

        Args:
            x (torch.Tensor): Input tensor of shape (batch_size, in_channels,
    depth, height, width).

        Returns:
            torch.Tensor: Output tensor of shape (batch_size, out_channels,
    depth-kernel_size+1, height-kernel_size+1, width-kernel_size+1).
        """
        x = self.conv(x)
        x = torch.tanh(x)
        x = torch.clamp(x, min=self.clamp_min, max=self.clamp_max)
        x = torch.nn.functional.gelu(x)
        return x
```

The task is made up of a computationally intensive function (`3D convolution`) and a series of lightweight functions (`tanh, clamp, gelu`).

The model implements a correct solution in the first turn by fusing the kernels, but it only achieves a speedup of **1.06x** over the reference implementation.

```
    import torch
from torch.utils.cpp_extension import load_inline
import math

class ModelNew(torch.nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, clamp_min,
    clamp_max):
        super(ModelNew, self).__init__()
```

```
    self.clamp_min = clamp_min
    self.clamp_max = clamp_max

    # Initialize convolution weights and bias
    self.weight = torch.nn.Parameter(torch.empty(out_channels, in_channels,
kernel_size, kernel_size, kernel_size))
    self.bias = torch.nn.Parameter(torch.empty(out_channels))
    # Weight initialization following PyTorch's Conv3d default
    torch.nn.init.kaiming_uniform_(self.weight, a=math.sqrt(5))
    fan_in, _ = torch.nn.init._calculate_fan_in_and_fan_out(self.weight)
    bound = 1 / math.sqrt(fan_in)
    torch.nn.init.uniform_(self.bias, -bound, bound)

    # Define the fused convolution and activation CUDA kernel
    conv_activation_source = """
    #include <torch/extension.h>
    #include <cuda_runtime.h>
    #include <math.h>

    __global__ void conv_activation_kernel(const float* input, const float*
weights, const float* bias, float* output,
                                           int N, int C_in, int D, int H, int W,
int C_out, int kD, int kH, int kW,
                                           float clamp_min, float clamp_max) {
        int idx = blockIdx.x * blockDim.x + threadIdx.x;
        const int W_out = W - kW + 1;
        const int H_out = H - kH + 1;
        const int D_out = D - kD + 1;
        const int total_elements = N * C_out * D_out * H_out * W_out;
        if (idx >= total_elements) return;

        // Decompose index into output coordinates
        int w_out = idx % W_out;
        int h_out = (idx / W_out) % H_out;
        int d_out = (idx / (W_out * H_out)) % D_out;
        int f = (idx / (W_out * H_out * D_out)) % C_out;
        int n = idx / (C_out * D_out * H_out * W_out);

        float sum = 0;
        for (int c = 0; c < C_in; ++c) {
            for (int kd = 0; kd < kD; ++kd) {
                for (int kh = 0; kh < kH; ++kh) {
                    for (int kw = 0; kw < kW; ++kw) {
                        // Input tensor index calculation
                        int d_in = d_out + kd;
                        int h_in = h_out + kh;
                        int w_in = w_out + kw;
                        int input_idx = n * C_in * D * H * W +
                                        c * D * H * W +
                                        d_in * H * W +
                                        h_in * W +
                                        w_in;
                        // Weight tensor index calculation
                        int weight_idx = f * C_in * kD * kH * kW +
                                         c * kD * kH * kW +
                                         kd * kH * kW +
                                         kh * kW +
                                         kw;
                        sum += input[input_idx] * weights[weight_idx];
                    }
                }
            }
        }
        sum += bias[f];
```

```
        // Apply activations sequentially
        sum = tanh(sum);
        sum = fmaxf(clamp_min, fminf(sum, clamp_max));
        {
            float inner = sum + 0.044715f * sum * sum * sum;
            inner *= 0.79788456f; // sqrt(2/pi) approximation
            sum = sum * 0.5f * (1.0f + tanh(inner));
        }

        // Output tensor index calculation
        int output_idx = n * C_out * D_out * H_out * W_out +
                         f * D_out * H_out * W_out +
                         d_out * H_out * W_out +
                         h_out * W_out +
                         w_out;
        output[output_idx] = sum;
    }

    torch::Tensor conv_activation_cuda(torch::Tensor input, torch::Tensor
weights, torch::Tensor bias,
                                       float clamp_min, float clamp_max) {
        // Dimension extraction
        int N = input.size(0);
        int C_in = input.size(1);
        int D = input.size(2);
        int H = input.size(3);
        int W = input.size(4);
        int C_out = weights.size(0);
        int kD = weights.size(2);
        int kH = weights.size(3);
        int kW = weights.size(4);

        // Output dimensions
        int D_out = D - kD + 1;
        int H_out = H - kH + 1;
        int W_out = W - kW + 1;

        // Create output tensor
        auto output = torch::empty({N, C_out, D_out, H_out, W_out},
input.options());

        // Launch kernel
        int total_elements = N * C_out * D_out * H_out * W_out;
        const int block_size = 256;
        const int grid_size = (total_elements + block_size - 1) / block_size;
        conv_activation_kernel<<<grid_size, block_size>>>(
            input.data_ptr<float>(), weights.data_ptr<float>(),
bias.data_ptr<float>(),
            output.data_ptr<float>(), N, C_in, D, H, W, C_out, kD, kH, kW,
            clamp_min, clamp_max
        );
        return output;
    }
    """

    conv_activation_header = """
    torch::Tensor conv_activation_cuda(torch::Tensor input, torch::Tensor
weights, torch::Tensor bias,
                                       float clamp_min, float clamp_max);
    """

    # Load the CUDA kernel
    self.conv_activation = load_inline(
        name="conv_activation",
        cpp_sources=conv_activation_header,
```

```
            cuda_sources=conv_activation_source,
            functions=["conv_activation_cuda"],
            verbose=True
        )

    def forward(self, x):
        return self.conv_activation.conv_activation_cuda(x, self.weight, self.bias,
    self.clamp_min, self.clamp_max)
```

On turn 2 it attempts to improve on the existing implementation, but fails due to a compilation error.

On turn 3, it attempts to modify the loop order to improve coalesced memory access, but this actually worsens performance, as the kernel now exhibits a speedup of **0.61x**. The model attempts this strategy again at turn 4, and is able to order the loops in a way that increases memory coalescing: this brings up the performance to **1.19x**. At turn 5 and 6, the model attempts some minor optimizations (such as precomputing certain indices, or deleting intermediate variables), which however has no measurable effect in performance.

On turn 7, the model figures out a surprising optimization: it records the convolution's kernel size as a compile time constant, which allows it to use `# pragma unroll` to unroll the 3 loops needed to apply the convolution. This results in a significant speedup of **1.93x** over baseline.

```
from torch.utils.cpp_extension import load_inline
import math

class ModelNew(torch.nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, clamp_min,
    clamp_max):
        super(ModelNew, self).__init__()
        self.clamp_min = clamp_min
        self.clamp_max = clamp_max

        # Initialize convolution weights and bias
        self.weight = torch.nn.Parameter(torch.empty(out_channels, in_channels,
    kernel_size, kernel_size, kernel_size))
        self.bias = torch.nn.Parameter(torch.empty(out_channels))
        # Weight initialization following PyTorch's Conv3d default
        torch.nn.init.kaiming_uniform_(self.weight, a=math.sqrt(5))
        fan_in, _ = torch.nn.init._calculate_fan_in_and_fan_out(self.weight)
        bound = 1 / math.sqrt(fan_in)
        torch.nn.init.uniform_(self.bias, -bound, bound)

        # Define the fused convolution and activation CUDA kernel with optimizations
        kernel_size_val = kernel_size
        conv_activation_source = f"""
        #include <torch/extension.h>
        #include <cuda_runtime.h>
        #include <math.h>

        #define KERNEL_SIZE {kernel_size_val}
        #define KERNEL_D {kernel_size_val}
        #define KERNEL_H {kernel_size_val}
        #define KERNEL_W {kernel_size_val}

        __global__ void conv_activation_kernel(
            const float* __restrict__ input, const float* __restrict__ weights,
    const float* __restrict__ bias,
            float* __restrict__ output,
            int N, int C_in, int D, int H, int W, int C_out,
            float clamp_min, float clamp_max) {{
                int idx = blockIdx.x * blockDim.x + threadIdx.x;
                const int W_out = W - KERNEL_W + 1;
                const int H_out = H - KERNEL_H + 1;
                const int D_out = D - KERNEL_D + 1;
                const int total_elements = N * C_out * D_out * H_out * W_out;
```

```
            if (idx >= total_elements) return;

            int w_out = idx % W_out;
            int h_out = (idx / W_out) % H_out;
            int d_out = (idx / (W_out * H_out)) % D_out;
            int f = (idx / (W_out * H_out * D_out)) % C_out;
            int n = idx / (C_out * D_out * H_out * W_out);

            float sum = 0.0;

            // Precompute constants for GELU
            const float sqrt_2_over_pi = 0.79788456f;
            const float approx_coeff = 0.044715f;

            for (int c = 0; c < C_in; ++c) {{
                // Compute input channel base pointer
                const int input_c_offset = n * C_in * D * H * W + c * D * H * W;
                const float* input_c = input + input_c_offset;

                // Compute center position in the input channel
                const int center_offset = d_out * H * W + h_out * W + w_out;
                const float* center = input_c + center_offset;

                // Weight base for current output channel f and input channel c
                const int weight_base = f * C_in * KERNEL_D * KERNEL_H *
KERNEL_W + c * KERNEL_D * KERNEL_H * KERNEL_W;

                #pragma unroll
                for (int kd = 0; kd < KERNEL_D; ++kd) {{
                    #pragma unroll
                    for (int kh = 0; kh < KERNEL_H; ++kh) {{
                        #pragma unroll
                        for (int kw = 0; kw < KERNEL_W; ++kw) {{
                            const int offset = kd * H * W + kh * W + kw;
                            const float input_val = center[offset];
                            const int w_off = weight_base + kd * KERNEL_H *
KERNEL_W + kh * KERNEL_W + kw;
                            const float weight_val = weights[w_off];

                            sum += input_val * weight_val;
                        }}
                    }}
                }}
            }}

            sum += bias[f];

            // Apply activations with optimized order
            {{
                float tanh_val = tanhf(sum);
                float clamped = fmaxf(clamp_min, fminf(tanh_val, clamp_max));
                float inner = clamped + approx_coeff * clamped * clamped *
clamped;
                inner *= sqrt_2_over_pi;
                sum = clamped * 0.5f * (1.0f + tanhf(inner));
            }}

            // Output index calculation
            const int output_base = n * C_out * D_out * H_out * W_out +
                                    f * D_out * H_out * W_out +
                                    d_out * H_out * W_out +
                                    h_out * W_out;
            output[output_base + w_out] = sum;
        }}
```

```
    torch::Tensor conv_activation_cuda(torch::Tensor input, torch::Tensor
weights, torch::Tensor bias,
                                       float clamp_min, float clamp_max) {{
        int N = input.size(0);
        int C_in = input.size(1);
        int D = input.size(2);
        int H = input.size(3);
        int W = input.size(4);
        int C_out = weights.size(0);

        int D_out = D - KERNEL_D + 1;
        int H_out = H - KERNEL_H + 1;
        int W_out = W - KERNEL_W + 1;

        auto output = torch::empty({{N, C_out, D_out, H_out, W_out}},
input.options());

        int total_elements = N * C_out * D_out * H_out * W_out;
        const int block_size = 256;
        const int grid_size = (total_elements + block_size - 1) / block_size;

        conv_activation_kernel<<<grid_size, block_size>>>(
            input.data_ptr<float>(), weights.data_ptr<float>(),
bias.data_ptr<float>(),
            output.data_ptr<float>(), N, C_in, D, H, W, C_out,
            clamp_min, clamp_max
        );
        return output;
    }}
    """

    conv_activation_header = """
    torch::Tensor conv_activation_cuda(torch::Tensor input, torch::Tensor
weights, torch::Tensor bias,
                                       float clamp_min, float clamp_max);
    """

    # Load the CUDA kernel with fast math optimization
    self.conv_activation = load_inline(
        name="conv_activation",
        cpp_sources=conv_activation_header,
        cuda_sources=conv_activation_source,
        functions=["conv_activation_cuda"],
        extra_cuda_cflags=['-use_fast_math'],
        verbose=True
    )

def forward(self, x):
    return self.conv_activation.conv_activation_cuda(x, self.weight, self.bias,
self.clamp_min, self.clamp_max)
```

In its final turn, the model attempts a more advanced implementation that further parallelizes the computation across kernels before performing a warp-level reduction. However, it fails to implement the strategy correctly, due to applying the reduction across the wrong axis. We do note the model has shown success in implementing complex warp reductions in several other tasks.